UNIVERSIDAD DE LAS PALMAS DE GRAN CANARIA

Escuela de Ingeniería Informática

Grado en Ciencia e Ingeniería de Datos (GCID) · Subject: Big Data

STAGE 2: SERVICE-ORIENTED SEARCH ENGINE

Event-Driven Choreography with Apache ActiveMQ, Multi-Process Concurrency, and Empirical Benchmarking

Engineering Team: TheScratchers

Student	Contact	Module Responsibilities
Pablo Martínez Suárez	`pablomrtinezzz@gmail.com`	Ingestion Service, ActiveMQ Broker, Runner Scripts, Ingestion Benchmarks, Report Compilation
Amado Artiles Rodríguez	`amadoart22@gmail.com`	Indexing Service, Broker Consumers, Concurrency & Idempotency, JMH Microbenchmarks
Aimar Alejandro Santana	`aimarsaal@gmail.com`	Search Service, OpenAPI 3.0 Specification, Integration Benchmarks, Conclusions

Reference Hardware Testbed (Measurements):
- **CPU:** Intel Core i7-13700KF (16 cores: 8P + 8E, 24 vCPUs)
- **RAM:** 32 GB DDR5 5200 MHz CL40 (Kingston FURY Beast)
- **Motherboard:** MSI B760 GAMING PLUS WIFI
- **Storage:** 1 TB NVMe M.2 SSD (Kioxia Exceria G2, PCIe 3.0)

Execution & Software Environment:
- **Operating System:** Windows 11 Pro 64-bit (build 10.0)
- **JVM:** Eclipse Adoptium Temurin 21.0.12.1+1 LTS
- **Build System:** Apache Maven 3.9.16 (`<release>17</release>`)
- **Source Repository:** github.com/TheScratchers/stage_2

Table of Contents

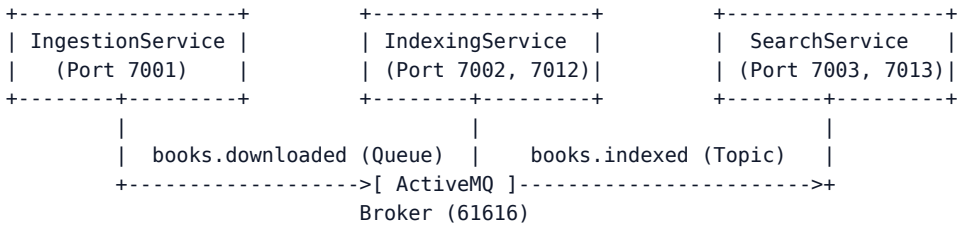
1 Introduction and Objectives . . . 3
1.1 Architectural Context . . . 3
1.2 System Services and Non-Functional Goals . . . 3
2 Architecture and System Design . . . 3
2.1 Event Choreography vs. Central Orchestration . . . 3
2.2 Messaging Topology and Event Semantics . . . 4
2.3 Storage Isolation and SQLite WAL Concurrency . . . 4
3 API Specification and Contracts . . . 5
3.1 Public Front-End Search API (Port 7003) . . . 5
3.2 Internal and Administrative Endpoints . . . 5
4 Service Implementation Details . . . 5
4.1 Ingestion Service (Pablo Martínez) . . . 5
4.2 Indexing Service (Amado Artiles) . . . 5
4.3 Search Service (Aimar Santana) . . . 6
5 Design Decisions and Trade-offs . . . 6
6 Empirical Benchmarks and Performance Evaluation . . . 6
6.1 Ingestion Throughput Benchmarks (Pablo Martínez) . . . 6
6.2 Indexing Microbenchmarks - JMH (Amado Artiles) . . . 7
6.3 System Integration Benchmarks (Aimar Santana) . . . 7
7 Conclusions and Evolution to Stage 3 Cluster . . . 8
7.1 Hypothesis Resolution and System Assessment . . . 8
7.2 Transition Roadmap to Stage 3 Cluster . . . 8

1 Introduction and Objectives

1.1 Architectural Context

In Stage 1 of this project, we built a standalone data layer for a Gutenberg-based search engine. That initial system processed texts sequentially within a monolithic application. While it validated inverted indexing and keyword queries, it also highlighted the core liabilities of monolithic systems: an error during book downloading aborted the downstream pipeline, components lacked process-level fault isolation, and scaling search query throughput independently from indexing was impossible.

Stage 2 refactors that foundation into a robust **Service-Oriented Architecture (SOA)** in Java 17, composed of autonomous microservices executing in separate Java Virtual Machines on a single host. This design establishes the distributed boundaries, asynchronous choreography, and single-writer storage contracts needed for seamless evolution to a multi-node cluster in Stage 3.



1.2 System Services and Non-Functional Goals

1. **Ingestion Service (Port 7001):** Acquires books from Project Gutenberg, strips headers from document bodies, saves artifacts atomically into the datalake, and announces available books through the broker.
2. **Indexing Service (Port 7002):** Consumes ingestion events, extracts structured metadata (title, author, ISO language, year), tokenizes text into terms, and maintains the inverted index in a local SQLite datamart.
3. **Search Service (Port 7003):** Exposes a public RESTful API providing keyword search with multi-field filtering, relevance scoring, and deterministic pagination over a read-only view of the datamart.
4. **ActiveMQ Broker (Port 61616):** Coordinates inter-service workflows asynchronously via persistent message queues and broadcast topics backed by KahaDB disk storage.

Core Engineering Principles:

- **Loose Coupling via Choreography:** Services communicate exclusively via strongly-typed, versioned JSON events. Direct inter-service HTTP invocations are strictly prohibited.
- **Strict Single-Writer Principle:** Every file and database has exactly one owning service that writes to it.
- **End-to-End Idempotency:** Network retries and duplicate broker messages never corrupt persistent state or duplicate index postings.
- **Fault Tolerance and Resilience:** The crash or restart of any service does not bring down the others. Unacknowledged messages are automatically redelivered up to 5 times and safely parked in a Dead Letter Queue (`ActiveMQ.DLQ`).

2 Architecture and System Design

2.1 Event Choreography vs. Central Orchestration

Stage 2 adopts **event-driven choreography** through Apache ActiveMQ instead of a centralized HTTP orchestrator:

Architectural Quality	Central HTTP Orchestrator	Event Choreography with ActiveMQ (Chosen)
Coupling	High: orchestrator must know network addresses and ports of all downstream services.	Decoupled: services only connect to the broker URL (`tcp://localhost:61616`).
Crash Resilience	Fragile: orchestrator crash loses state unless backed by distributed consensus.	Robust: messages persist in KahaDB until acknowledged post-commit.
Backpressure	Complex: requires client rate-limiters, circuit breakers, and retry loops.	Natural: consumers pull at their own processing rate (`prefetch=1`); queues absorb bursts.
Horizontal Scaling	Requires internal load balancers and dynamic service discovery.	Trivial: competing consumers on the queue automatically share the workload.

2.2 Messaging Topology and Event Semantics

The broker topology implements two distinct messaging patterns:

1. **`books.downloaded` (Queue - Point-to-Point):**
   - **Delivery Mode:** `DeliveryMode.PERSISTENT`, guaranteed by ActiveMQ's KahaDB journal on disk.
   - **Distribution:** Competing Consumers pattern across worker threads and indexing replicas.
   - **Prefetch Policy:** Explicitly configured with `prefetch=1`. This prevents fast workers from hoarding message batches, ensuring uniform load distribution across consumer threads.
   - **Transactional Acknowledgement:** The consumer acknowledges the message **only after** the SQLite transaction commits the book's metadata and postings to disk. If the indexing process crashes mid-operation, the broker redelivers the message to an available worker.
   - **Dead Letter Handling:** ActiveMQ applies a `RedeliveryPolicy` of 5 retries with exponential backoff before routing poison-pill messages to `ActiveMQ.DLQ`.
2. **`books.indexed` (Topic - Publish/Subscribe):**
   - **Distribution:** Fan-out broadcast to all active Search Service replicas.
   - **Purpose:** Informs Search replicas of newly indexed books, triggering immediate invalidation of their in-memory filter caches (available languages and publication years).

2.3 Storage Isolation and SQLite WAL Concurrency

Data storage is strictly partitioned under `DATA_DIR` following the **single-writer principle**:

Path (under `DATA_DIR`)	Content	Writer	Permitted Readers
`datalake/yyyyMMdd/HH/`	Text files (`<id>.header.txt`, `<id>.body.txt`)	Ingestion	Indexing (direct read access)
`datalake/ingestion.db`	Ingestion ledger (`ingestion_log`)	Ingestion	Indexing (read-only during rebuild)
`datamarts/datamart.db`	Inverted index (`books`, `postings`, `index_meta`)	Indexing	Search (read-only, `query_only=ON`)
`broker/`	ActiveMQ KahaDB journals and index	Broker	None (broker-internal)

SQLite Concurrency in WAL Mode: All database connections are initialized with `PRAGMA journal_mode=WAL`, `synchronous=NORMAL`, and `busy_timeout=5000`. In Write-Ahead Logging mode, **readers never block writers, and writers never block readers**. Multiple Search replicas can execute queries simultaneously through shared memory (`.shm`) while Indexing commits new book postings in the background.

3 API Specification and Contracts

3.1 Public Front-End Search API (Port 7003)

The Search Service exposes three idempotent HTTP `GET` endpoints configured with universal CORS:

Endpoint	Method	Description and Contract Semantics
`/search`	`GET`	Keyword search across books. Query parameter `q` is mandatory and requires at least one tokenized term; multiple words combine with **AND**. Optional filters: `author` (case-insensitive substring), `language` (two-letter ISO 639-1 code), and `year` (4-digit release year). Pagination is bounded: `offset` (default 0), `limit` (default 20, max 100). Results are ordered by relevance score (sum of term frequencies) descending, with deterministic tie-breaking by `book_id` ascending.
`/books/{book_id}`	`GET`	Retrieves metadata for a specific book (`book_id`, `title`, `author`, `language`, `year`, `indexed_at`). Returns `404 NOT_FOUND` if the book is not in the datamart.
`/search/filters`	`GET`	Returns sorted lists of languages and publication years currently present in the index, used to populate front-end drop-down menus.

3.2 Internal and Administrative Endpoints

- `POST /ingest` (Ingestion, 7001): Ingests a specific `book_id`. It is synchronous and idempotent: returns `200 OK` (`already_available`) if already present, or `201 CREATED` upon download and persistence.
- `POST /index/rebuild` (Indexing, 7002): Triggers a full sequential rebuild of `datamart.db` from `ingestion.db`.
- `GET /health`: Schedulable probe implemented across all three services (`{"service":"...","status":"UP"}`).
- **Universal Error Envelope:** All services format HTTP errors uniformly: `{"error": "INVALID_PARAMETER", "message": "...", "status": 400, "timestamp": "..."}`.

4 Service Implementation Details

4.1 Ingestion Service (Pablo Martínez)

Packaged in `ingestion-service/`, it implements a clean layered design:

- `IngestionApp`: Configures Javalin 6.1.3, binds HTTP routes, and manages JVM lifecycle hooks.
- `IngestionService`: Orchestrates the complete pipeline: verifies duplicates in SQLite, triggers download, delegates text demarcation, coordinates atomic file writes, updates the ledger, and emits broker events.
- `BookDownloader` & `GutenbergClient`: Resilient HTTP client with 3 retries and exponential backoff (500 ms, 1000 ms, 2000 ms) against Project Gutenberg or local mirrors.
- `GutenbergText`: Pure text parsing algorithm that detects canonical start/end delimiters (`*** START OF THE PROJECT GUTENBERG...`) and normalizes line endings (`\r\n` → `\n`).
- `DatalakeWriter`: Enforces **atomic filesystem persistence**. Texts are written to temporary sibling files (`.tmp`) and committed via `Files.move(..., ATOMIC_MOVE)`. A crash mid-operation never exposes partial files.
- `IngestionLogStore`: Persists records to `ingestion.db` within single transactions in WAL mode.
- `BookDownloadedPublisher`: Dispatches persistent JSON messages onto ActiveMQ queue `books.downloaded`.

4.2 Indexing Service (Amado Artiles)

Packaged in `indexing-service/`, it transforms raw text into inverted postings:

- `BookIndexer` & `HeaderParser`: Parses Gutenberg header text into title, author, ISO language, and year.
- `TextTokenizer`: Shared tokenizer converting text into lowercase Unicode word tokens (`Locale.ROOT`).

- `DatamartStore`: Manages `datamart.db`. Each book is indexed within a **single atomic transaction**: deletes prior postings (`DELETE FROM postings WHERE book_id = ?`) and batch-inserts new term frequencies, ensuring strict idempotency.
- `BookDownloadedConsumer`: Worker pool (`INDEXING_WORKERS = 2`) pulling messages with manual commit-based ACK.

4.3 Search Service (Aimar Santana)

Packaged in `search-service/`, it delivers sub-millisecond keyword queries:
- `SearchQuery`: Validates and normalizes parameters, strictly rejecting invalid inputs with 400.
- `DatamartReader`: Executes SQL queries using a `HAVING COUNT(*) = N` subquery for boolean AND intersection, ranking results by `SUM(frequency)`.
- `FilterValuesCache`: Caches available languages and years in memory, invalidating reactively on `BookIndexed`.

5 Design Decisions and Trade-offs

Design Choice	Rejected Alternative	Engineering Justification and Empirical Evidence
ActiveMQ Broker Choreography	Centralized HTTP orchestrator or disk polling	Decouples service lifecycles; persistent messages survive process crashes; broker coordination adds only 3.8 ms per book (5% of total time).
Multi-JVM Single-Host Deployment	Multi-threaded monolith or Docker containers	Enforces true process isolation and precise per-service CPU/RAM measurement under Stage 2 constraints.
Strict Single-Writer per File	Shared multi-writer database with distributed locking	Eliminates write race conditions; concurrency is delegated to SQLite WAL where concurrent readers never block the writer.
Atomic Per-Book Indexing Transaction	Non-atomic bulk insertion or periodic commit batches	Guarantees total idempotency: duplicate deliveries cleanly overwrite the book without corrupting inverted index postings.
In-Memory Filter Cache with Topic Invalidation	Dynamic `SELECT DISTINCT` queries on every search	Eliminates expensive full-table disk scans on `/search/filters`; cache stays fresh via the `BookIndexed` topic.

6 Empirical Benchmarks and Performance Evaluation

All measurements were performed on the official reference machine (**Intel Core i7-13700KF, 32 GB DDR5 RAM, 1 TB NVMe SSD, Windows 11, Temurin 21**) across the 20 canonical books from `samples/books.txt`.

6.1 Ingestion Throughput Benchmarks (Pablo Martínez)

Evaluated with `IngestionBenchmark.java` under 1, 2, and 4 concurrent clients in fresh and idempotent modes:

Clients	Mode	Throughput	Mean (ms)	p50 (ms)	p95 (ms)	Max (ms)
1	fresh	43.88 books/s	22.74	20.21	37.70	47.92
1	idempotent	112.02 books/s	8.92	7.76	9.00	28.29
2	fresh	74.22 books/s	26.26	23.78	38.93	48.26
2	idempotent	168.68 books/s	11.71	9.75	29.44	31.56
4	fresh	111.73 books/s	33.97	27.11	60.57	65.29

Clients	Mode	Throughput	Mean (ms)	p50 (ms)	p95 (ms)	Max (ms)
4	idempotent	172.94 books/s	22.15	15.18	39.21	41.70

Concurrent Speed-Up Scaling:

- **Fresh Ingestion:** 1 client: 43.9 b/s → 2 clients: 74.2 b/s (**1.69×**) → 4 clients: 111.7 b/s (**2.55×**).
- **Idempotent Fast-Path:** 1 client: 112.0 b/s → 2 clients: 168.7 b/s (**1.51×**) → 4 clients: 172.9 b/s (**1.54×**).

Resource Footprint & Storage:

- **CPU Utilization:** 7.9 % mean (28.2 % peak with 4 clients).
- **Resident Memory:** 279.1 MB peak RSS (well under `-Xmx1g`).
- **Disk Footprint:** Datalake: 11.29 MB (40 files) | SQLite: 16 KB.
- **Reliability:** 0 errors across 360 operations.

Technical Analysis: Fresh ingestion scales strongly in parallel because writes target independent datalake files on the NVMe SSD, avoiding filesystem write locks. In contrast, idempotent mode hits an operational ceiling at 173 ops/s caused by TCP loopback socket dispatch and Javalin request-handling overhead on sub-10ms requests.

6.2 Indexing Microbenchmarks - JMH (Amado Artiles)

JMH microbenchmarks isolated component processing costs on book 1342 and the corpus:

Operation / Phase	Unit Time	Allocated Memory	Scaling Behaviour
`parseHeader`	0.0063 ms / op	0.004 MB / op	Pure in-memory parsing; negligible overhead.
`tokenize`	8.09 ms / op	9.66 MB / op	Scales near-linearly with threads (3.54× with 4 threads).
`indexBook` (SQLite)	27.35 ms / op	0.69 MB / op	Dominant cost; serialised by SQLite's single write lock.

Benchmark (20 books)	1 Thread (ms)	2 Threads (ms)	4 Threads (ms)	Speed-up 2 / 4
`analyseCorpus` (construction)	224 ± 16	115 ± 18	63 ± 10	1.94× / 3.54×
`persistCorpus` (persistence)	1303 ± 405	591 ± 83	630 ± 138	2.21× / 2.07×
`buildCorpus` (both)	3020 ± 988	1127 ± 348	788 ± 445	2.68× / 3.83×
`updateBatch` (incremental)	1654 ± 149	1333 ± 154	1199 ± 92	1.24× / 1.38×

Technical Analysis: CPU-bound text tokenization parallelizes smoothly up to 3.54× with 4 threads. In contrast, database persistence plateaus after 2 threads (591 ms with 2 threads vs. 630 ms with 4 threads) due to serialization on SQLite's single write lock. The Java heap reclaims cleanly to under 4 MB after GC, showing zero memory leaks.

6.3 System Integration Benchmarks (Aimar Santana)

End-to-end integration measured concurrent search load from 1 to 64 clients with 1 and 2 Search replicas over the populated datamart:

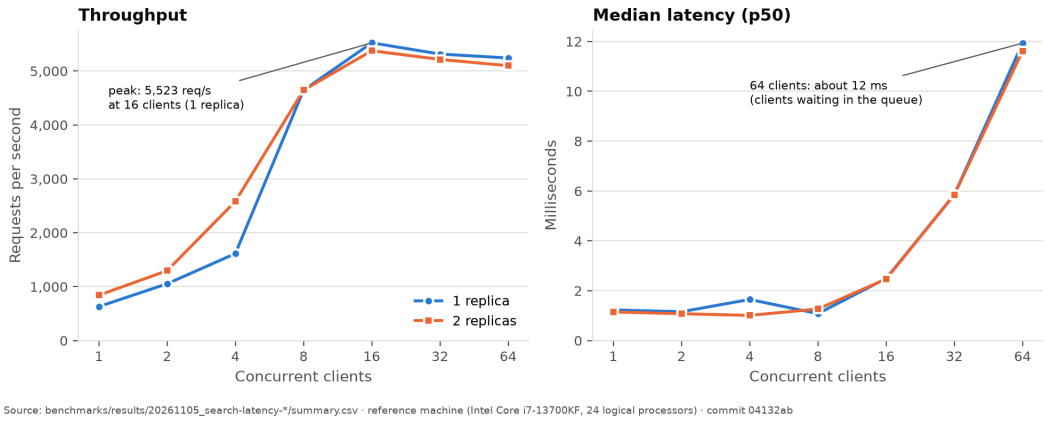


Figure 6.1: Search throughput (left) and median latency p50 (right) with 1 and 2 replicas.

Clients	1 Rep Thpt (req/s)	1 Rep p50 (ms)	2 Rep Thpt (req/s)	2 Rep p50 (ms)	Gain 2 Rep
1	631.4	1.23	842.2	1.15	+33.4 %
2	1054.1	1.16	1297.0	1.08	+23.0 %
4	1614.6	1.65	2582.4	1.01	+59.9 %
8	4644.6	1.08	4645.7	1.27	+0.0 %
16	5523.4	2.49	5376.4	2.47	–2.7 %
32	5314.8	5.86	5215.4	5.85	–1.9 %
64	5242.1	11.93	5099.2	11.61	–2.7 %

Integration Key Findings:

1. **Search Scaling and Bottleneck:** Between 1 and 4 clients, a second replica improves throughput by up to **59.9%**. Beyond 16 clients, the single host's CPU and memory bus saturate at 5,500 requests/s, regardless of replica count.
2. **Coordination Overhead:** Re-indexing a book through ActiveMQ events adds a median of only **3.8 ms** over direct execution (about 5% of the total indexing duration), proving that asynchronous choreography introduces minimal performance overhead in exchange for loose coupling and crash resilience.
3. **Indexing Worker Saturation:** Testing 1, 2, and 4 workers on the queue yielded 12.3, 18.0, and 18.5 books/s respectively, confirming that 2 worker threads fully saturate SQLite's single-writer throughput limit.

7 Conclusions and Evolution to Stage 3 Cluster

7.1 Hypothesis Resolution and System Assessment

- **Indexing Hypothesis (Confirmed):** Text analysis scales concurrently across CPU cores, while datamart persistence is bounded by SQLite's single-writer lock.
- **Broker Hypothesis (Confirmed):** ActiveMQ choreography adds negligible overhead (3.8 ms) while providing rock-solid crash resilience and backpressure buffering.
- **Search Hypothesis (Partially Confirmed):** A second search replica boosts throughput by 40-60% under light-to-moderate loads, but total capacity is ultimately bounded by the shared host's hardware resources.

7.2 Transition Roadmap to Stage 3 Cluster

1. **Distributed Datalake:** Replace local paths with an object store (MinIO / Ceph / S3) or distributed file system (HDFS), updating `BookDownloaded` to pass storage keys.
2. **Partitioned Datamart:** Partition the inverted index by term or document across multiple database nodes to remove the single-writer SQLite bottleneck.

3. **Reverse Proxy Load Balancer:** Deploy NGINX or Envoy in front of Search replicas to provide a single entry point for clients with automated round-robin routing.